Agent Dashboard

How every number on **/agent** is produced

This report traces each figure on the call-center agent dashboard back to the exact query and the exact SQL that produces it. It answers the two questions that started it: **are these numbers counted at lead creation, at status update, or at assignment**, and **which set of leads are they drawn from**.

Scope	Route **/agent** — the call-center agent dashboard
Frontend	frontend/src/pages/agent/Dashboard.tsx
Backend	backend/src/routes/lead.routes.ts, backend/src/lib/leadRelease.ts
Method	Static read of the source: 5 React Query calls traced into their Prisma queries
Verdict	The page mixes **two different counting universes**. Four real defects found; two of them change numbers on screen.

The one-sentence answer

Phase 1 (Confirmation) is counted by status update — rebuilt from the rows you wrote into `lead_status_history`, not from the leads table. **Phase 2 (Livraison), « À traiter » and « Paniers convertis » are counted by current assignment** — leads where `assignedAgentId` is still you. That difference is deliberate, and it is why two panels on the same page can disagree about the same lead.

1. What the page actually loads

The dashboard fires five independent queries. None of them counts rows on a page — every endpoint returns a server-side aggregate that is computed separately from the row limit, so raising or lowering « limit » never changes a number the agent reads.

Query	Endpoint	What it feeds	Universe
leadStatsQuery	GET /leads?withStats=true	**All of Phase 1** — the donut, the 8 tiles, the 3 activity boxes — plus the « En attente d'envoi » counter	History
todoQuery	GET /leads?status=ASSIGNED,NEW	« À traiter » counter and the « Actions Requises » list at the bottom	Assignment
deliveryQuery	GET /leads/livraison	**All of Phase 2** plus the « Colis en cours » counter	Assignment
availableQuery	GET /leads/available	« Disponibles » counter	Shared pool
cartsQuery	GET /leads/abandoned-carts	« Paniers convertis » counter and the PANIERS CONVERTIS tile	Assignment

Every one of them receives the same four period parameters (`dateFrom, dateTo, dateField, dateType`), so in principle the whole page describes one window. In practice two of the five ignore part of it — see section 9.

There is no polling and no websocket on this page. React Query is configured with `staleTime: 5 min`, so numbers can be up to five minutes old until the agent presses the refresh button in the hero.

Dashboard.tsx:329-362 (queries) · main.tsx:12 (staleTime) · Dashboard.tsx:377-383 (refreshAll)

2. The two counting universes

This is the concept that explains almost every surprise on the page.

Universe A — History (what you did)

Source: the `lead_status_history` table. Every time anyone changes a lead's status, a row is written recording the new status, the timestamp, and `changedBy` — the user who did it.

Universe B — Assignment (what you currently hold)

Source: the `leads` table, column `assignedAgentId`. This is a single pointer: a lead sits on exactly one agent's plate at a time.

> **Why the split exists at all**
>
> A background cron reclaims leads an agent worked but did not convert — NO_REPLY, CALL_LATER, CANCEL_REASON_PRICE and others go back to the shared pool after 1–2 hours. When it does that it **wipes** `assignedAgentId`. If the agent's own performance figures were counted off the leads table, every lead the cron took back would erase the claim, the calls and the outcome all at once. `changedBy` in the history table survives that, so Phase 1 is built there instead.

leadRelease.ts:16-25 (RELEASE_TRACKED_STATUSES) · lead.routes.ts:34-55 (AGENT_TRACKED_ACTIONS)

The practical consequence, stated plainly:

- A lead the cron took back **keeps** its Phase 1 credit forever, but **disappears** from « À traiter », from Phase 2 and from « Paniers convertis ».
- A lead another agent force-claimed from you behaves the same way: your actions stay counted, the lead itself leaves your assignment-based panels.
- So Phase 1 and Phase 2 are **not** two views of one set of leads. Phase 1 is your work log; Phase 2 is your current inventory.

3. Phase 1 in detail — counted by status update

For a `CALL_CENTER_AGENT`, the backend computes the usual `groupBy(status)` over the leads table and then **throws the result away**, replacing it with figures rebuilt from history. The overwrite is literal:

```
// lead.routes.ts:618-619
byStatus  = agentByStatus;      // <- the leads-table group-by is discarded
scopeTotal = lastActionPerLead.size;
```

The query it is rebuilt from:

```
// lead.routes.ts:550-575
SELECT leadId, newStatus, lead.status
FROM   lead_status_history
WHERE  changedBy = <you>
  AND  (notes IS NULL OR notes NOT LIKE 'Systeme :%')     -- exclude cron rows
  AND  <period, see section 9>
ORDER BY createdAt DESC                                    -- newest first
```

Why the cron rows are excluded

When the cron releases a lead it writes a history row stamped with **the agent's own id**, so the lead's timeline reads as one continuous story. But the system letting a lead go is not something the agent did. Those rows are prefixed « `Système :` » and filtered out here. Without that filter two things would break at once: the agent would be credited with an action for every lead that timed out on them, and the release row — being the newest — would become their « last action », pulling the lead out of the outcome they actually recorded.

leadRelease.ts:76-87 (SYSTEM_NOTE_PREFIX) · lead.routes.ts:551-558

The three tallies built from those rows

Field	Definition	Answers the question
`byAction[X]`	Count of **every** history row whose newStatus is X. Ringing one number three times gives three NO_REPLY.	How much work did I do?
`byLead[X]`	Count of **distinct leads** that ever received X from you.	How many customers did I mark X at least once?
`agentByStatus[X]`	Each lead filed **once**, under your **last** action on it. Rows are read newest-first, so the first row seen for a lead wins.	Where did each of my leads end up?

The three boxes above the donut read straight off this:

Box on screen	Backend field	Meaning
Actions	`totalActions`	Number of history rows — every status change you recorded
Leads traités	`leadsWorked`	Distinct leads you touched

Box on screen	Backend field	Meaning
Réclamés	`claimed = byLead.ASSIGNED`	Distinct leads you pulled out of the pool — **including ones since handed back**

Zero-filling

Eight statuses are always present in the response even when you never used them (ASSIGNED, CALL_LATER, NO_REPLY, CONFIRMED, WRONG_ORDER, CANCEL_REASON_PRICE, CANCEL_ORDER, PUSHED_TO_DELIVERY). That is why a tile you never lit renders as a **0** rather than vanishing from the grid.

lead.routes.ts:46-55, 610-616

4. Worked example — the same day, counted four ways

Nothing explains this better than one concrete day. Suppose that today you do exactly this:

Time	Lead	What you do	History row written
09:00	#101	Claim it from the pool	ASSIGNED
09:05	#101	Call — nobody answers	NO_REPLY
09:10	#102	Claim it from the pool	ASSIGNED
09:12	#102	Call — customer confirms	CONFIRMED
09:30	#102	Push the parcel to Coliaty	PUSHED_TO_DELIVERY
10:40	#101	*cron releases it back to the pool*	(« Système : » row — ignored)
10:45	#101	You claim it again	ASSIGNED
10:50	#101	Call again — still nobody	NO_REPLY
11:00	#103	Claim it — no call made yet	ASSIGNED

That is **8 counted history rows** across **3 distinct leads** (the cron row does not count). Here is what each mechanism makes of it.

The three activity boxes

Actions	Leads traités	Réclamés
8	**3**	**3**
8 rows written	#101, #102, #103	you claimed all three at least once

Mode « Par action » — every status change counted

Tile	Value	Where it comes from
ASSIGNED	4	3 first claims + 1 re-claim of #101
NO_REPLY	2	you rang #101 twice
CONFIRMED	1	#102 — the push is **not** folded in, in this mode

Tile	Value	Where it comes from
CALL LATER / WRONG ORDER / CANCEL x2	0	never used today

Traités (denominator) = 2 + 1 = **3** · **Taux** = 1 / 3 = **33.3 %** · donut total = 3 + 4 = **7**

Mode « Par lead » — each lead filed under its last action

Lead	Last action	Filed under	Why
#101	NO_REPLY (10:50)	NO_REPLY	newest row wins; the cron row is excluded
#102	PUSHED_TO_DELIVERY	CONFIRMED	pushed leads are folded back into CONFIRMED
#103	ASSIGNED	ASSIGNED	claimed, no outcome recorded yet

Traités (denominator) = 1 NO_REPLY + 1 CONFIRMED = **2** — ASSIGNED is excluded · **Taux** = 1 / 2 = **50 %** · donut total = 2 + 1 = **3**

> **Both rates are correct**
> 33.3 % and 50 % are two honest answers to two different questions. **Par action** scores the calls: you made three result-calls and one of them landed. **Par lead** scores the customers: you finished with two customers and converted one. Neither is a bug — but an agent comparing today's rate with yesterday's must stay in the same mode, because the toggle is not persisted and resets to **Par lead** on every reload.

What the assignment-based panels say about the same day

Panel	Value	Why it differs
À traiter	1	only #103 is still `assignedAgentId = you` with status ASSIGNED. #101 was released to the pool; #102 is pushed.
En attente d'envoi	0	no lead's last action is CONFIRMED-and-not-yet-pushed — #102 already left
Colis en cours / Phase 2	1 parcel	#102 has an order and is still assigned to you
Disponibles	pool size	unrelated to your work — and #101 has just re-entered this number

So on this single day, Phase 1 says you worked **3** leads and Phase 2 shows **1** parcel, while « À traiter » says **1**. All three are right. They are counting different things.

5. The two special rules inside Phase 1

Rule 1 — the PUSHED_TO_DELIVERY override (backend)

When filing a lead under its last action, the backend checks the lead's **current** status first. If it is already `PUSHED_TO_DELIVERY`, the lead is filed as pushed no matter what the newest history row says.

```
// lead.routes.ts:594-597
lastActionPerLead.set(
  row.leadId,
  row.lead?.status === 'PUSHED_TO_DELIVERY' ? 'PUSHED_TO_DELIVERY' : row.newStatus
);
```

The reason is historical: the Coliaty hand-off only started writing its own history row recently, so without this override every lead pushed before that change would still read as merely CONFIRMED.

Rule 2 — folding pushed leads back into CONFIRMED (frontend)

The dashboard shows six outcome tiles and PUSHED_TO_DELIVERY is not one of them. So in **Par lead** mode the frontend adds the pushed count into the CONFIRMED tile:

```
// Dashboard.tsx:437-447
const queued = counts.CONFIRMED || 0;
const pushed = countMode === 'actions' ? 0 : counts.PUSHED_TO_DELIVERY || 0;
value = queued + pushed;
hint  = pushed > 0 ? `${queued} en attente - ${pushed} envoyes` : o.hint;
```

It is folded **in** rather than shown alongside, so a lead always lands in exactly one slice of the donut. In **Par action** mode there is no folding: a confirm and a push are two separate things you did, and both are already counted separately.

This is also why the CONFIRMED tile carries a two-part hint such as « 3 en attente · 12 envoyés » instead of its usual subtitle.

Rule 3 — two different denominators, on purpose

Name	Formula	Used by	Why
`treatedTotal`	sum of the **6 outcomes** only	the « Taux » badge	ASSIGNED is a state, not a result. Counting un-called leads as failures would drag the rate down for work you have not had a chance to do yet.
`pipelineTotal`	treatedTotal + ASSIGNED	donut centre, legend bars, tile percentages	the eye compares against the whole pipeline, including what is still in progress

Consequence to be aware of: the percentage printed on the CONFIRMED tile and the percentage in the « Taux » badge **will not match** whenever you hold any ASSIGNED lead. That is by design, but it reads as a contradiction on screen.

Dashboard.tsx:479-483

6. The five counters in the hero, one by one

Counter	Exact rule	Universe
Disponibles	`COUNT(leads)` where status = AVAILABLE **and** assignedAgentId IS NULL **and** no order attached, restricted to the influencers/vendors you are assigned to (and, if the admin narrowed your account, to those specific products).	Shared pool
À traiter	`pagination.total` of leads where `assignedAgentId = you` and status in (ASSIGNED, NEW), minus leads that already carry a Coliaty package code.	**Assignment**
En attente d'envoi	`byStatus.CONFIRMED` — and remember byStatus has been replaced by the history-derived map. So: leads whose **last action by you** was CONFIRMED and which are not already pushed (rule 1 moves those out).	**History**
Colis en cours	Phase 2 groups pipeline + transit + issue. Base scope: leads where `assignedAgentId = you` **and** an order exists. Counted on the **order's** status, not the lead's.	**Assignment**

Counter	Exact rule	Universe
Paniers convertis	Abandoned carts with a `convertedLeadId` whose resulting lead is **currently assigned to you**. The sub-label « sur N paniers » is the whole visible cart set, which excludes any cart whose phone number already exists as a lead anywhere in the system.	**Assignment**

lead.routes.ts:828-833 · :302-307, :339-346 · :618 · :1206-1207 · :4762-4782, :4796-4811

> **A trap in « Paniers convertis »**
> Because a converted cart is only counted while its lead is still assigned to you, this number **goes down on its own** when the cron releases that lead back to the pool. It is the only figure on the page that can decrease without anyone doing anything.

7. Phase 2 — Livraison Coliaty

Phase 2 is a completely separate endpoint with its own logic. Its base scope for an agent is:

```
// lead.routes.ts:1203-1207
baseWhere = {
  order:           { isNot: null },   // the lead became a real order
  assignedAgentId: <you>,             // and is still yours
}
```

Every counter in Phase 2 is then built by walking that scope row by row and bucketing on `order.status`. The 24 delivery statuses are grouped into five families purely on the frontend:

Group	Label on screen	Statuses in it
pipeline	Préparation	PENDING, PUSHED_TO_DELIVERY, NEW_PARCEL, WAITING_PREPARATION, ENCORE_PREPARED, PREPARED, WAITING_PICKUP
transit	En transit	PICKED_UP, SENT, RECEIVED, DISTRIBUTION, PROGRAMMER, PROGRAMMER_AUTO
issue	Incidents	POSTPONED, NOANSWER, ERR, INCORRECT_ADDRESS
done	Livré	DELIVERED
return	Retours & annulations	RETURNED, REFUSE, CANCELED, CANCELED_BY_SELLER, CANCELED_BY_SYSTEM

« Colis en cours » = pipeline + transit + issue. « Taux » = DELIVERED / total parcels in the window. Any status the backend returns that is not in the table above still renders — it falls back to a grey box with the raw key lower-cased, so nothing is silently dropped.

Dashboard.tsx:120-159, 486-514

8. What the period filter really filters

The « Filtrer par date de » toggle sends one value that is read differently by each endpoint. This is where most of the confusion on this page lives.

Mode	Label on screen	For Phase 1 it means	For the leads list it means
`updatedAt`	Mise à jour	the **history row's own timestamp** — i.e. **when you performed the action**. Not `lead.updatedAt`.	`lead.updatedAt`
`createdAt`	Création	`lead.createdAt` — when the lead **arrived**, whenever you happened to work it	`lead.createdAt`

```
// lead.routes.ts:560-567
if (targetDateField === 'createdAt') historyWhere.lead = { is: { createdAt: range } };
else                                 historyWhere.createdAt = range;   // the action itself
```

Two genuinely different questions, and an agent needs both. « What did I get through today » is the work — **Mise à jour**. « How are today's arrivals doing » is the intake — **Création**. A status moved this morning on a lead from last week belongs to the first and not to the second.

The presets

Preset	Range sent	Note
Aujourd'hui	from = today 00:00, to = *empty*	open-ended on purpose, so it stays correct as the day goes on
Hier	from = yesterday 00:00, to = yesterday 23:59	the only preset with a closing bound. It is 23:59 and not the next midnight, because the server treats a bound given to the minute as covering that whole minute — midnight would leak the first sixty seconds of today into the count.
7 jours / 30 jours	from = N-1 days ago 00:00, to = *empty*	rolling, includes today
Tout	both empty	unbounded

The default on load is **Aujourd'hui** with mode **Mise à jour**. The two `datetime-local` fields underneath accept anything else to the minute, and any range that matches no preset simply lights no chip.

9. Drill-downs

Clicking any of the six outcome tiles opens the leads page with the same window:

```
/agent/leads?historyStatus=CONFIRMED&dateField=updatedAt&dateFrom=...&dateTo=...
```

The backend treats `historyStatus` as the one and only reason to reopen an agent's history scope. It then re-applies the identical « `Système :` » exclusion and the identical date semantics, so the list you land on counts the same rows the tile counted — including leads the cron has since taken away from you.

```
// lead.routes.ts:378-390
statusHistory: { some: {
  changedBy: <you>,
  newStatus: <historyStatus>,
  OR: [{ notes: null }, { notes: { not: { startsWith: 'Systeme :' } } }],
  ...(range && field === 'updatedAt' ? { createdAt: range } : {}),
}}
```

> **One drill-down that under-reports**
> The CONFIRMED tile links to `historyStatus=CONFIRMED`, but its displayed value in **Par lead** mode **includes the folded-in PUSHED_TO_DELIVERY leads** (rule 2). So a tile reading 15 can open a list of 3. The tile is right; the link is too narrow.

10. Defects found

Four real issues. Two of them put wrong numbers on the screen.

D1 · Phase 2 silently ignores the date mode

Severity: numbers wrong on screen. The parcel **list** honours `dateField`, but the **stats** block — which is everything Phase 2 displays — hard-codes the order's creation date:

```
// lead.routes.ts:1436-1442 + :1517
const inWindow = (at) => { ... };            // built from `createdRange` only
...
if (!inWindow(row.order?.createdAt)) continue;   // <- always order.createdAt
```

Switching the toggle to **Mise à jour** changes every figure on the page *except* Phase 2, which keeps answering « parcels **created** in this window ». Nothing on screen says so.

D2 · « Paniers convertis » corrupts the donut arithmetic

Severity: numbers wrong on screen. The cart row is pushed into `phase1Rows`, so it renders as a donut slice, a legend bar and a percentage — but it is deliberately **not** part of `pipelineTotal`, which is the denominator every one of those uses.

Three visible consequences:

- The donut's slices sum to **more** than the number printed in its centre.
- The tile's percentage can exceed **100 %**. With pipelineTotal = 3 and 2 converted carts, the tile prints **66.7 %** of a total it is not in.
- Worse: converting a cart **also creates a lead assigned to you**, which is already counted under ASSIGNED or CONFIRMED. The same customer is drawn twice in one chart.

Dashboard.tsx:466-483 (row construction and pipelineTotal)

D3 · « Disponibles » ignores the date mode too

Severity: inconsistent, low impact. `/leads/available` has no `dateField` handling at all — it only ever applies `createdAt`. Arguably correct for a pool of unclaimed leads (they have no meaningful update), but it means the toggle is a no-op on two of the five hero counters rather than one.

lead.routes.ts:879-881

D4 · « À traiter » queries a status that can never match

Severity: dead code. The query asks for `status=ASSIGNED,NEW`, but the agent scope forces `assignedAgentId = you`, and a lead with status NEW is by definition unassigned. The NEW half never matches anything. Harmless today, but the « Actions Requises » list below renders a « Nouveau » badge for a case that cannot occur.

Dashboard.tsx:337, 1048 · lead.routes.ts:302-307

Recommended order of fixing

D2 first — it is the only one that produces a visibly impossible number (a percentage over 100 and a donut that does not add up), and the fix is a few lines: either add the cart row to `pipelineTotal`, or drop it from `confirmationDist` and leave it as a tile only. **D1** second — one line, pass `targetDateField` into `inWindow` instead of hard-coding `order.createdAt`.

11. Cheat sheet

One table, every number on the page.

Number on screen	Counted by	Drawn from	Honours date mode?
Disponibles	current pool state	leads: AVAILABLE, unassigned, in your scope	No — createdAt only
À traiter	assignment	leads: assignedAgentId = you	Yes
En attente d'envoi	status update	history: last action = CONFIRMED	Yes
Colis en cours	assignment	orders on leads still assigned to you	No — order.createdAt only
Paniers convertis	assignment	carts whose lead is assigned to you	Yes
Actions	status update	history: row count	Yes
Leads traités	status update	history: distinct leads	Yes
Réclamés	status update	history: distinct leads ever ASSIGNED by you	Yes
Phase 1 tiles — Par lead	status update	history: each lead under its last action	Yes
Phase 1 tiles — Par action	status update	history: every row	Yes
Taux de confirmation	status update	CONFIRMED / the 6 outcomes	Yes
Phase 2 tiles	assignment	order.status on your assigned leads	No
Taux de livraison	assignment	DELIVERED / all your parcels	No
Actions Requises list	assignment	leads: assignedAgentId = you	Yes

Three sentences to remember

- **Phase 1 is your work log.** It is rebuilt from what you wrote into the status history, so it never loses work when a lead leaves you.
- **Phase 2, À traiter and Paniers convertis are your current inventory.** They read the leads table, so they lose a lead the moment the cron or another agent takes it.
- **« Mise à jour » means the time of the action, not the lead's updatedAt** — and two of the five counters ignore it entirely.

Source files traced: `frontend/src/pages/agent/Dashboard.tsx`, `backend/src/routes/lead.routes.ts`, `backend/src/lib/leadRelease.ts`, `frontend/src/main.tsx`. Line references throughout are against the working tree at the time of the scan.